INTRODUCTION TO PYTHON
UNIT 3: DATA STRUCTURES (STRINGS, LISTS, TUPLES, DICTS & SETS)
Curriculum: Lovely Professional University | School of Electronics & Communication Engineering
Reference: Python Programming: Using Problem Solving Approach by Reema Thareja

>> 3.1 Sequences and String Manipulation

Python sequences are ordered collections of items indexed by integers:
* Sequence Indexing: Zero-based positive indexing (0 to N-1) from left; negative indexing (-1 to -N) from right.
* Slicing Syntax: sequence[start:stop:step]. Omitting start defaults to 0; omitting stop defaults to end; step=-1 reverses sequence.
* Strings (str): Immutable sequences of Unicode characters. Modifying a character causes TypeError; transformations return new string instances.
* Common String Methods: s.strip(), s.split(sep), s.join(iterable), s.upper(), s.lower(), s.replace(old, new), s.find(sub), s.count(sub), s.startswith(pre), s.endswith(suf).

>> 3.2 Lists: Mutable Ordered Collections

* List Creation: list_var = [elem1, elem2, ...] or list(iterable). Heterogeneous elements permitted.
* In-Place Mutation Methods: .append(x) (adds to tail in O(1)), .extend(iterable), .insert(index, x), .remove(val) (removes first occurrence), .pop(index=-1) (removes and returns element), .clear(), .sort(key=None, reverse=False), .reverse().
* Built-in Sequence Functions: len(), min(), max(), sum(), sorted(), any(), all().
* List Comprehensions: [expression for item in iterable if condition]. Provides concise, readable, and C-optimized syntax for transforming lists.

>> 3.3 Tuples: Immutable Heterogeneous Records

* Definition: Immutable sequences enclosed in parentheses: t = (1, "A", 3.14). Singleton tuple requires trailing comma: t_single = (42,).
* Immutability Rationale: Data integrity (protects constants from accidental alteration), hashability (tuples can serve as keys in dictionaries and elements in sets, whereas lists cannot).
* Tuple Packing & Unpacking: a, b, c = (10, 20, 30). Variable swapping: x, y = y, x. Extended unpacking: first, *middle, last = numbers.

>> 3.4 Dictionaries: High-Speed Key-Value Hash Tables

* Structure: Unordered (ordered by insertion since Python 3.7) mappings of unique hashable keys to arbitrary values: d = {key: value}.
* Performance: O(1) average-time complexity for lookups, insertions, and deletions via internal hash table addressing.
* Dictionary Methods: .keys(), .values(), .items(), .get(key, default) (avoids KeyError on missing key), .pop(key), .update(dict2), .setdefault(k, d).
* Dictionary Comprehensions: {key_expr: value_expr for item in iterable if condition}.

>> 3.5 Sets: Mathematical Set Operations

* Definition: Unordered collections of unique, immutable elements enclosed in curly braces: s = {1, 2, 3}. Empty set must be initialized as set() (not {} which creates a dict).
* Mathematical Operations: Union (|), Intersection (&), Difference (-), Symmetric Difference (^), Subset (<=), Superset (>=).

[PROGRAM 1: Telecommunication Serial Packet Parser]

Problem Context: Parse a raw ASCII serial telemetry packet structured as "$HEADER,DEVICE_ID,VOLTAGE,CURRENT,TEMP*CHECKSUM", validate packet integrity, and extract numerical sensor data.
Python Source Code:

```
def parse_telemetry_packet(raw_packet):
# Check start and end markers
if not raw_packet.startswith('$') or '*' not in raw_packet:
return False, 'Malformed packet framing'

# Strip framing and split payload from checksum
packet_core = raw_packet[1:] # remove $
payload, claimed_chk_hex = packet_core.split('*')

# Verify XOR Checksum (NMEA 0183 protocol standard)
calculated_chk = 0
for char in payload:
calculated_chk ^= ord(char)

claimed_chk = int(claimed_chk_hex, 16)
if calculated_chk != claimed_chk:
return False, f'Checksum mismatch! Calc: {calculated_chk:#04x}, Claimed:
{claimed_chk:#04x}'

# Tokenize CSV fields
```

```
tokens = payload.split(",")
telemetry_dict = {
'header': tokens[0],
'device_id': tokens[1],
'voltage_v': float(tokens[2]),
'current_a': float(tokens[3]),
'temp_c': float(tokens[4])
}
return True, telemetry_dict

# Test packet: payload="TLM,NODE_01,12.45,1.82,43.5"
test_packet = '$TLM,NODE_01,12.45,1.82,43.5*24'
success, result = parse_telemetry_packet(test_packet)
if success:
print('Packet Successfully Parsed:')
for k, v in result.items():
print(f' {k:<12} : {v}')
else:
print('Packet Error:', result)
```

[PROGRAM 2: Pure-Python Matrix Multiplier using List Comprehensions]

Problem Context: Multiply two 2D matrices A (m x k) and B (k x n) using nested list comprehensions without importing external libraries.

Python Source Code:

```
def matrix_multiply(A, B):
rows_A, cols_A = len(A), len(A[0])
rows_B, cols_B = len(B), len(B[0])

if cols_A != rows_B:
raise ValueError(f'Incompatible shapes: ({rows_A}x{cols_A}) and ({rows_B}x{cols_B})')

# Dot product: sum(A[i][k] * B[k][j] for k in range(cols_A))
C = [[sum(A[i][k] * B[k][j] for k in range(cols_A))
for j in range(cols_B)]
for i in range(rows_A)]
return C

A = [[1, 2, 3],
[4, 5, 6]]
B = [[7, 8],
[9, 1],
[2, 3]]
```

```
result = matrix_multiply(A, B)
print('Matrix A (2x3):', A)
print('Matrix B (3x2):', B)
print('Product C (2x2):')
for row in result:
print(' ', row)
```

[PROGRAM 3: Electronic Component Inventory Manager using Dict & Tuples]

Problem Context: Manage electronic components in a laboratory inventory with quantities, bin locations, and minimum threshold alerts.

Python Source Code:

```
# Key: Part Number | Value: (Description, Quantity, Unit Price, Bin Location)
inventory = {
'RES_10K': ('Resistor 10k 0.25W', 1500, 0.05, 'BIN_A1'),
'CAP_100U': ('Capacitor 100uF 25V', 45, 0.35, 'BIN_B2'),
'IC_LM741': ('Op-Amp uA741 DIP8', 12, 0.75, 'BIN_C4'),
'DIODE_1N4007': ('Diode 1000V 1A', 800, 0.08, 'BIN_A3')
}

def check_restock(inv, threshold=50):
restock_list = []
for part, (desc, qty, price, bin_loc) in inv.items():
if qty < threshold:
restock_list.append((part, desc, qty, bin_loc))
return restock_list

low_stock = check_restock(inventory, threshold=50)
print('=== LOW STOCK ALERT (< 50 units) ===')
for part, desc, qty, bin_loc in low_stock:
print(f'[ALERT] {part:<12} ({desc}): Current Qty = {qty} (Located in {bin_loc})')
```

[PROGRAM 4: RF Frequency Channel Overlap Detection using Sets]

Problem Context: Identify shared channels and co-channel interference risks between two wireless communication gateways using Python set operations.

Python Source Code:

```
gateway_A_channels = {1, 3, 6, 8, 11, 14, 18, 22}
gateway_B_channels = {6, 7, 11, 15, 18, 24, 28}

# 1. Total unique channels utilized
all_channels = gateway_A_channels | gateway_B_channels
```

```
# 2. Interfering Co-Channels (Intersection)
interfering_channels = gateway_A_channels & gateway_B_channels

# 3. Channels exclusive to Gateway A
exclusive_A = gateway_A_channels - gateway_B_channels

print(f'Gateway A Active Channels : {sorted(list(gateway_A_channels))}')
print(f'Gateway B Active Channels : {sorted(list(gateway_B_channels))}')
print(f'Co-Channel Overlap (Interf) : {sorted(list(interfering_channels))}')
print(f'Clean Channels Exclusive to A: {sorted(list(exclusive_A))}')
```